Network Protocol Simulator (TCP/IP & UDP)

Overview

This project simulates a network environment with configurable packet loss, latency, and corruption. It features:

- **TCP** : full state machine (CLOSED → LISTEN → SYN_SENT → ESTABLISHED → FIN_WAIT → CLOSED), stop-and-wait reliability, retransmission on timeout, and graceful 4-way teardown
- **UDP** : lightweight fire-and-forget datagram delivery
- **Channel** : simulated physical medium with configurable loss rate, delay, and corruption
- **Wireshark-style logger** : timestamped packet trace for every transmitted frame
- **Discrete-event engine** : priority-queue scheduler that processes events in chronological order

Prerequisites

Requirement	Minimum version
C++ compiler (g++ or clang++)	C++17
GNU Make	3.8+
Linux / macOS / WSL	—

No external libraries are required. The project relies solely on the C++ standard library.

Installation

Clone the repository :

```
git clone https://github.com/titou4n/Network-Protocol-Simulator-TCP-IP-and-UDP.git
```

Go to the folder :

```
cd Network-Protocol-Simulator-TCP-IP-and-UDP
```

Build the project :

```
make
```

To clean :

```
make clean       # removes object files
 make fclean     # removes everything including binary
```

To rebuild :

```
make re
```

Configuration

All network parameters are set in `simulation/Simulator.cpp` inside `Simulator::run()`:

```
double loss_rate   = 0.1;       // Packet loss probability  (0.0 = no loss, 1.0 = total loss)
double delay       = 100.0;     // Simulated latency in milliseconds
double corruption = 0.05;       // Packet corruption probability
bool   wireshark    = true;     // Enable/disable Wireshark-style logging
```

Payload and chunk size for TCP data transfer :

```
std::string payload   = "Network Protocol Simulator with TCP/IP and UDP";
size_t      chunkSize = 10;  // bytes per TCP segment
```

TCP timeout before retransmission :

```
TCP tcp_client = TCP(channel);          // default: 500ms
TCP tcp_client = TCP(channel, 300);  // custom:  300ms
```

Channel Parameters :

Parameter	Type	Description
`loss_rate`	`double` [0.0–1.0]	Probability of a packet being silently dropped
`delay`	`double` (ms)	Fixed one-way propagation delay
`corruption`	`double` [0.0–1.0]	Probability of a packet being corrupted in transit
`wireshark`	`bool`	Enables timestamped packet logging to stdout

Running the Simulator

Using the simulator engine

```
./main
```

This runs the full discrete-event simulation: UDP test, TCP handshake, data transfer, and teardown.

Example output

```
========== SIMULATION START ==========

 [t=0ms] ========== UDP TEST ==========
 [TCP] SERVER: Listening...
 [TCP] CLIENT: Sending SYN
 [TCP] [NODE 2] SYN received -> sending SYN-ACK
 [TCP] [NODE 1] SYN-ACK received -> sending ACK
 [TCP] [NODE 2] ACK received -> [CONNECTION ESTABLISHED]

 TCP_SEND [Network Pro]
 [TCP] [NODE 1] SEND seq=1
 [TCP] [NODE 2] DATA received seq=1 | SEND ACK

 ========== TCP DISCONNECT ==========
 [TCP] SERVER: CONNECTION CLOSED

 ========== SIMULATION END ==========
```

Usage

Sending a UDP datagram :

```
UDP udp;
UDPPacket packet(1, 2, "Hello UDP");
udp.send(packet, channel);
```

Establishing a TCP connection :

```
TCP tcp_client = TCP(channel);
TCP tcp_server = TCP(channel);

server.setTCP(&tcp_server);
client.setTCP(&tcp_client);

tcp_server.listen();
tcp_client.connect(client, server);

// Wait for full handshake (10s timeout)
auto deadline = std::chrono::steady_clock::now() + std::chrono::seconds(10);
while (!tcp_client.isConnected() || !tcp_server.isConnected())
{
    if (std::chrono::steady_clock::now() > deadline) break;
    tcp_client.checkTimeout(channel);
    tcp_server.checkTimeout(channel);
}
```

Sending data over TCP :

```
TCPPacket packet(client.getId(), server.getId(), "Hello TCP");
tcp_client.send(packet);

// Stop-and-wait: wait for ACK
auto deadline = std::chrono::steady_clock::now() + std::chrono::seconds(10);
while (tcp_client.isWaitingForAck())
{
    if (std::chrono::steady_clock::now() > deadline) break;
    tcp_client.checkTimeout(channel);
    tcp_server.checkTimeout(channel);
}
```

Closing a TCP connection :

```
tcp_client.disconnect(client.getId(), server.getId());

// Wait for full 4-way teardown
auto deadline = std::chrono::steady_clock::now() + std::chrono::seconds(10);
while (!tcp_client.isDisconnected() || !tcp_server.isDisconnected())
{
    if (std::chrono::steady_clock::now() > deadline) break;
    tcp_client.checkTimeout(channel);
    tcp_server.checkTimeout(channel);
}
```

Scheduling custom events :

```
Simulator sim;
sim.schedule(EventType::UDP_SEND,        0.0, 1, 2, "ping");
sim.schedule(EventType::TCP_LISTEN,      100.0, 2, 1);
sim.schedule(EventType::TCP_CONNECT,  101.0, 1, 2);
sim.schedule(EventType::TCP_SEND,        300.0, 1, 2, "hello");
sim.schedule(EventType::TCP_DISCONNECT,  500.0, 1, 2);
sim.run();
```

Project Architecture

```
Network-Protocol-Simulator-TCP-IP-and-UDP/
 |
 +-- core/
 |   +-- Channel.hpp / .cpp      # Simulated network medium
 |   +-- Node.hpp / .cpp         # Network endpoint
 |
 +-- packets/
 |   +-- Packet.hpp / .cpp       # Base packet
 |   +-- TCPPacket.hpp / .cpp  # TCP packet (seq, syn, ack, fin)
 |   +-- UDPPacket.hpp / .cpp # UDP packet (lightweight)
 |
 +-- protocols/
 |   +-- TCP.hpp / .cpp          # Full TCP state machine
 |   +-- UDP.hpp / .cpp          # Stateless UDP send/receive
 |
 +-- simulation/
 |   +-- Event.hpp / .cpp        # Event struct
 |   +-- Simulator.hpp / .cpp    # Priority-queue event scheduler
 |
 +-- tools/
 |   +-- Logger.hpp / .cpp       # Timestamped console logging
 |   +-- Metrics.hpp / .cpp      # Packet statistics
 |   +-- Wireshark.hpp / .cpp    # Wireshark-style trace output
 |
 +-- main.cpp                    # Entry point
 +-- Makefile
 +-- README.md
```

TCP State Machine :

```
CLOSED --listen()--> LISTEN
CLOSED --connect()--> SYN_SENT --SYN-ACK--> ESTABLISHED
LISTEN --SYN--> SYN_RECEIVED --ACK--> ESTABLISHED

ESTABLISHED --disconnect()--> FIN_WAIT_1
FIN_WAIT_1 --ACK--> FIN_WAIT_2 --FIN--> CLOSED
FIN_WAIT_1 --FIN--> CLOSED  (fast path, ACK dropped)
```

```
ESTABLISHED --FIN--> CLOSE_WAIT --> LAST_ACK --ACK--> CLOSED
```

Event Types

Event	Description
UDP_SEND	Send a UDP datagram
TCP_LISTEN	Put server into LISTEN state
TCP_CONNECT	Initiate TCP handshake from client
TCP_SEND	Send a TCP data segment
TCP_DISCONNECT	Initiate 4-way TCP teardown
SIM_END	Terminate the simulation loop

Troubleshooting

Simulation hangs indefinitely :

This happens when packet loss is 100% (loss_rate = 1.0). The timeout loops will hit the 10-second deadline and print:

```
[SIM] Connection timeout after 10s
```

Lower loss_rate to a value below 1.0.

Packets are never delivered :

Check that both nodes are registered with the channel before transmitting:

```
channel.add_node(client);
channel.add_node(server);
```

"Cannot connect: state=..." error :

The client TCP must be in CLOSED state before calling connect(). If you reuse a TCP object, reset it or create a new instance.

License

Authors

Project of Titouan SIMON for learning network protocols and systems programming in C++.